

ASSIGNMENT # 3

AI & DS

CS 2001 Data Structures

“Greatness isn’t given; it’s built. Challenge yourself daily, learn relentlessly, and strive for more because settling is the enemy of progress”

Instructions:

1. **Guidelines Compliance:** Any violation of the guidelines provided will result in a score of zero for the assignment.
2. **Softcopy Submission:** The assignment must be submitted in soft form on the GCR.
3. **No Email Submissions:** Under NO circumstances will submissions via email or any other way be accepted.
4. **Submission Deadline:** **March 18, 2026**
5. **Individual Work:** This is an individual assignment. Collaboration or copying will not be tolerated.
6. **Plagiarism:** **STRICT POLICY ON CHEATING THROUGH ANY SOURCE. ZERO MARKS WILL BE ASSIGNED IN ALL ASSIGNMENTS AND QUIZZES.**
7. **Identification Details:** Ensure that your Roll Number, Section, Name, program is written in the code files. **Rename the folder as ROLL-NUM_PROGRAM_SECTION (e.g., i230001_DS/AI_A) and compress the folder as a zip file. (e.g., i240001_DS/AI_A.zip).** Strictly follow this naming convention, otherwise marks will be deducted.
8. **Reference Material:** While you may consult textbooks or other resources for your understanding, ensure all work is done in your own words/code. Provide references at end if you have used anything from anywhere. **AI (GPT) is not allowed at all.**
9. **Programming Language:** The programming language allowed for this assignment is C++ only.
10. **Good Programming Practices:** Add comments in code for good programming practices. Make sure that code is neatly formatted and indented. Some marks are dedicated to this.
11. **Completeness and Correctness of Code:** Code should be complete and correct. If there is a syntax error in the code, zero marks will be awarded in that part of the assignment.
12. **Submission Guidelines:**
 - a. For each question in your assignment, make a separate .cpp/.h file e.g., for question 1, make q1.cpp, q1.h and so on. Each file that you submit must contain your name, student-id, and assignment number on top of the file in the comments (5 marks will be allocated for this).
 - b. Combine all your work in one folder. The folder must contain only .cpp files (no binaries, no exe files etc.).
 - c. The student is solely responsible for checking the final file for issues like corrupt file, viruses in the file, or mistakenly exe sent. If the teacher is not able to download the assignment from GCR, it will lead to zero marks in the assignment. Your teachers should be able to run your code on their laptop later to verify your outputs.

Best of Luck

Notes:

All the Outputs shown in the assignment are just for your understanding. There is no Hard and Fast rule to implement them as it is. More creativity and effort means More Marks.

Start early so that you can finish it on time.

Please ensure that only concepts covered in class are used for the assignment.

Question # 1 (Marks 50)

MrBeast Extreme Challenge Route Manager

MrBeast is hosting one of his largest survival challenges ever. Contestants must progress through a series of challenge zones spread across a massive indoor facility. Each zone contains a mini-game where contestants attempt to collect tokens while managing their energy levels.

Players move from one zone to another as they survive the challenges. However, the facility does not remain static. Zones may reorganize themselves depending on player performance, certain zones may remain locked until specific conditions are satisfied, and contestants may sometimes be forced to move backward if too many players are eliminated.

You are required to design a system that manages the challenge route and the players currently attempting each zone.

Challenge Route

The facility maintains a sequence of zones connected in both directions. Each zone represents a stage of the challenge.

Each zone must store the following information:

- Zone Name
- Difficulty Level (1–10)
- Locked / Unlocked Status
- Total Attempts made on the zone
- Reference to the next zone
- Reference to the previous zone
- Reference to a circular group of players currently inside the zone

Player Arena inside Each Zone

Players attempting a zone take turns in a circular rotation so that turns continue indefinitely until the zone is completed.

Each player stores:

- Player Name
- Current Energy
- Tokens Collected
- Elimination Status
- Reference to the next player in the rotation

Functional Requirements

1. Discovering New Zones

When a new zone is discovered, it must be inserted into the route according to its **difficulty level**.

If two zones have the same difficulty, the new zone must appear **after the zone that has recorded fewer attempts**.

2. Player Entry

Contestants may enter the challenge at any time.

When a player enters:

- They are added to the circular rotation of the current zone.
- If the player already exists in another zone, they must first be removed from their previous location.

3. Playing a Mini-Challenge

Players attempt the challenge one at a time according to the circular rotation.

During each attempt:

Energy decreases by a random value between **5 and 20**

Tokens increase by a random value between **0 and 3**

If a player's energy becomes **zero or negative**, the player is eliminated and removed from the rotation.

4. Zone Completion

A zone is completed when:

Total tokens collected $\geq$ Difficulty $\times$ 5

When this condition is satisfied:

- All surviving players move to the **next zone**
- The player rotation in the completed zone becomes empty

5. Route Reorganization

If a zone fails **three consecutive attempts**, the route reorganizes.

The zone swaps position with the zone immediately before it.

6. Locked Zones

Zones with difficulty **8 or higher** start locked.

They become accessible only when **at least three players survive the previous zone**.

7. Recovery Mode

If the number of active players drops below **two**, recovery mode activates.

Remaining players must move **back to the previous zone**.

8. Zone Removal

A zone may only be removed if:

- No players are currently inside it
- Its difficulty is **lower than the average difficulty of all zones**

Example Menu

Your program should be interactive and allow the user to perform operations through a menu similar to the following:

```
===== MrBeast Challenge Manager =====  
  
1. Discover New Zone  
2. Add Player to Current Zone  
3. Play Mini-Challenge Round  
4. Move Players to Next Zone  
5. Display Current Zone  
6. Display Full Challenge Route  
7. Remove Zone  
8. Exit  
  
Enter Choice:
```

Example Display Output

```
Challenge Route  
  
Zone: Desert Maze (Difficulty: 4)  
Players: Alex -> Sara -> Liam -> (back to Alex)  
  
Zone: Ice Tower (Difficulty: 7)  
Players: Noah -> Emma -> (back to Noah)  
  
Zone: Lava Bridge (Difficulty: 9) [Locked]  
Players: None
```

Marks Distribution

Component	Marks
Zone structure and route management	5
Player circular rotation implementation	5

Zone completion and token logic	6
Route reorganization and recovery mode	6
Display functions and menu system	3
Correct handling of edge cases	5
Demo	20
Total	50

Question # 2 (Marks 50)

The Circle of Betrayal

A new reality competition produced by Netflix places contestants inside a social strategy arena known as “**The Circle of Betrayal.**”

Contestants stand in a large circle and interact primarily with the two people next to them. While the formation appears simple, alliances constantly form and break. Betrayals occur unexpectedly and contestants are eliminated from the circle as the game progresses.

Your task is to build a system that simulates how contestants interact in this dynamic arena.

Contestant Circle

All contestants stand in a **continuous circular formation**.

This means:

- Every contestant has a neighbor before them
- Every contestant has a neighbor after them
- After the last contestant, the circle returns to the first contestant

Each contestant stores:

- Contestant Name
- Trust Score
- Betrayal Count
- Active / Eliminated Status
- Reference to the next contestant
- Reference to the previous contestant
- Reference to their alliance (if they lead one)

Alliance Groups

Contestants may secretly form alliances during the game. Each alliance maintains its members in a circular chain.

Each alliance member stores:

- Member Name
- Loyalty Level
- Reference to the next member in the alliance

Rules:

- A contestant may belong to **only one alliance**
- Alliance members must exist in the contestant circle

Functional Requirements

1. Adding Contestants

When a contestant joins the arena, they must be inserted at a position where the difference between their trust score and the trust scores of neighboring contestants does not exceed **20**.

If no such position exists, insert the contestant **before the starting contestant**.

2. Creating Alliances

A contestant may create an alliance and add members to it.

Alliance members must already exist in the main contestant circle.

3. Betrayal Event

During each round:

The contestant with the **lowest trust score** betrays someone.

The contestant immediately **clockwise** from them is eliminated and removed from the circle.

4. Alliance Collapse

If the eliminated contestant was leading an alliance:

- The alliance collapses
- All members remain in the arena
- Each member loses **10 loyalty points**

5. Alliance Revenge

Remaining alliance members retaliate against the contestant responsible for the elimination.

Each member reduces the eliminator's trust score based on their loyalty level.

6. Loyalty Decay

If the **average loyalty** of an alliance drops below **30**, the alliance collapses automatically.

7. Reverse Elimination Round

Every **five rounds**, eliminations occur in the opposite direction.

Instead of the clockwise neighbor, the **counter-clockwise contestant** is eliminated.

8. Contestant Migration

If a contestant's trust score drops below **15**, they relocate **two positions forward in the circle**.

The circle must remain intact after relocation.

9. Alliance Merging

If two alliance leaders become neighbors in the circle: The alliance with the **higher total loyalty** absorbs the other alliance.

Example Menu

```
===== Circle of Betrayal =====

1. Add Contestant
2. Create Alliance
3. Add Member to Alliance
4. Start Betrayal Round
5. Display Contestant Circle
6. Display Alliances
7. Next Round
8. Exit

Enter Choice:
```

Example Display Output

```
Contestant Circle

A ⇌ B ⇌ C ⇌ D ⇌ E ⇌ (back to A)

Alliances

Leader: B
Members: C -> D -> (back to C)

Leader: E
Members: A -> (back to A)
```

Marks Distribution

Component	Marks
Contestant circle implementation	10
Alliance circular structure	5
Betrayal and elimination logic	5
Alliance merge and collapse rules	5
Menu system and displays	2
Handling special conditions and edge cases	3
Demo	20
Total	50